


# CSE422 - Artificial Intelligence

## Spring-24

## Lab Project: House Price Prediction

Section: 09

Group No: 12

Group Members

22301061 - Faiaz Abrar Zaman Pronoy

21301474 - Md. Jahirul Islam Zishan

# Table of Contents


| Contents | Page No |
|---|---|
| Introduction | **3** |
| Dataset Description | **3-7** |
| Dataset Pre-Processing | **8-11** |
| Dataset Splitting | **11** |
| Model Training & Testing | **12-14** |
| Model Selection/Comparison Analysis | **14** |
| Conclusion | **14** |

**Estimating the Price of a House based on its Features & Location**

## Introduction:

One of the most important applications of machine learning is to predict the value of a target variable based on some input features. In this project, we aim to estimate the price of a house based on its features and location using three different machine learning algorithms: decision tree, linear regression and support vector machine (SVM). We will compare the performance of these algorithms on a real-world dataset of house prices. We will also explore how different hyperparameters and feature selection methods affect the accuracy and complexity of the models. By the end of this project, we will have a better understanding of how to apply machine-learning techniques to solve regression problems in practice.

## Dataset Description:

**Source:** Anna Montoya, DataCanary. (2016). House Prices - Advanced Regression Techniques. Kaggle. https://kaggle.com/competitions/house-prices-advanced-regression-techniques

**There are 78 features in the dataset, we selected these 20 features. Used Dataset:** https://drive.google.com/file/d/1xudsQYT4Au3vbpElsRicK4-ZMy2PWaRa/view?usp=drive_link

- Id: Serial of all rows.
- LotArea: Lot size in square feet
- Street: Type of road access
- Alley: Type of alley access
- Utilities: Type of utilities available
- Neighborhood: Physical locations within Ames city limits
- BldgType: Type of dwelling
- HouseStyle: Style of dwelling
- YearBuilt: Original construction date
- RoofStyle: Type of roof

● Foundation: Type of foundation

● Electrical: Electrical system

● GrLivArea: Above grade (ground) living area square feet

● FullBath: Full bathrooms above grade

● Bedroom: Number of bedrooms above the basement level

● Kitchen: Number of kitchens

● Fireplaces: Number of fireplaces

● GarageCars: Size of garage in car capacity

● SaleCondition: Condition of sale

● SalePrice - the property's sale price in dollars. This is the target variable that we are trying to predict.

**Classification or Regression:**

Based on the information we provided, predicting house prices based on these features would be a regression problem. Regression is used to predict a continuous numerical value, such as the price of a house. On the other hand, classification is used to predict a categorical value or class label. Since the goal is to predict the price of a house, which is a continuous numerical value, regression would be the appropriate choice.

**Data Points:** 1460

After applying the shape function, we find there are 1460 rows and 20 columns.

**Kind of Features:**

We have 10 Quantitative features and 10 Categorical features.

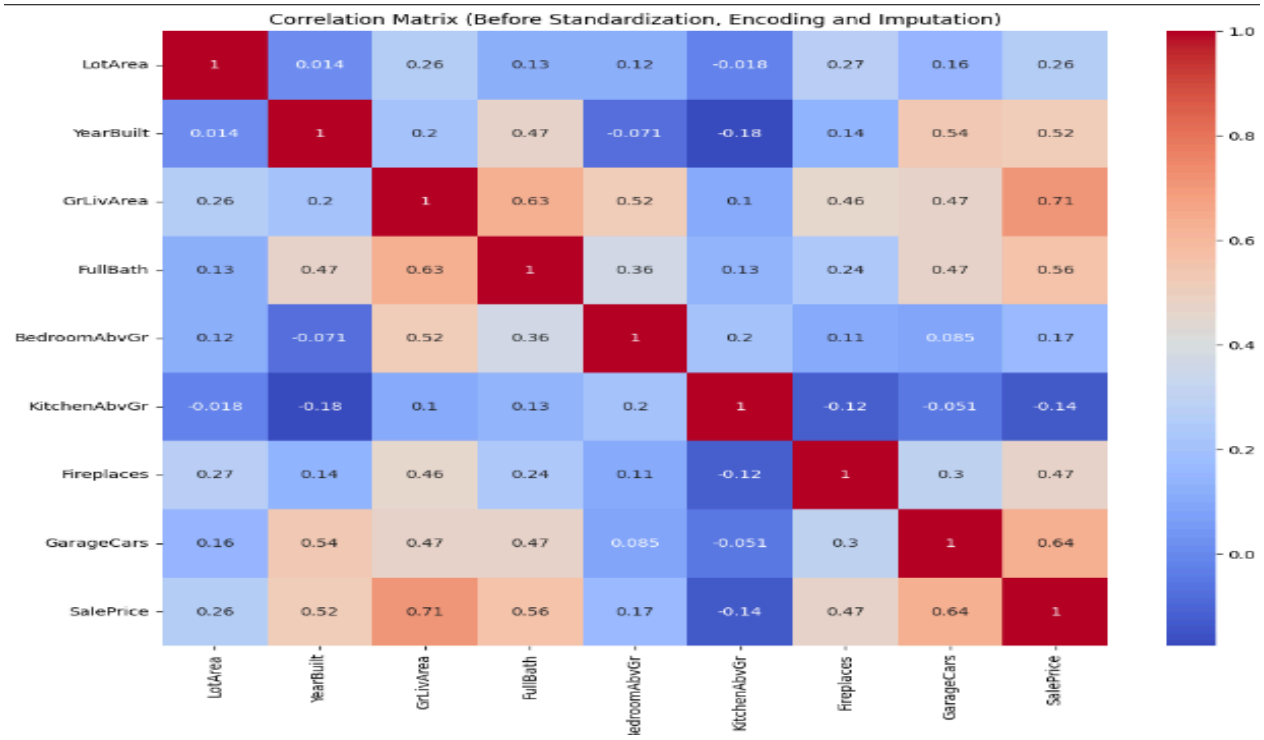
**Quantitative Features:** Id, LotArea, YearBuilt, GrLivArea, FullBath, BedroomAbvGr, KitchenAbvGr, Fireplaces, GarageCars, SalePrice.

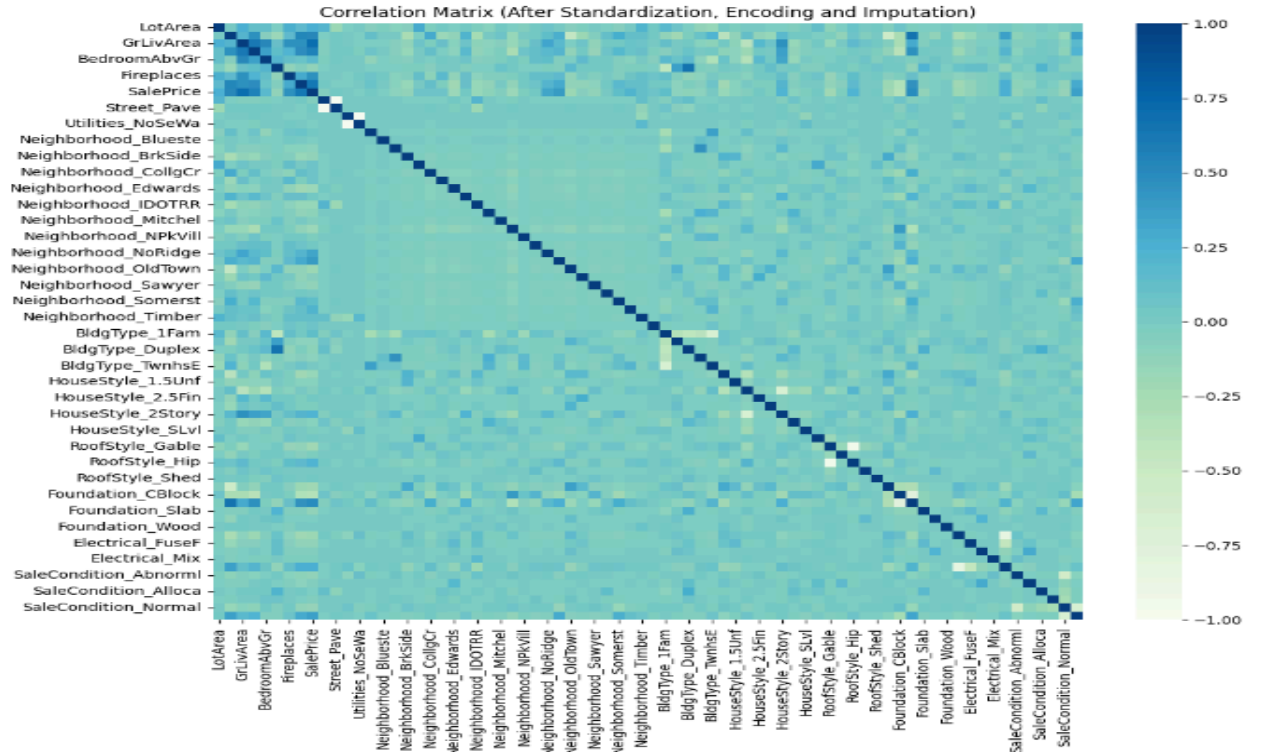
**Categorical Features:** Street, Alley, Utilities, Neighborhood, BldgType, HouseStyle, RoofStyle, Foundation, Electrical, SaleCondition.



| # | Column | Non-Null Count | Dtype |
|---|---|---|---|
| 0 | Id | 1460 non-null | int64 |
| 1 | LotArea | 1459 non-null | float64 |
| 2 | Street | 1460 non-null | object |
| 3 | Alley | 91 non-null | object |
| 4 | Utilities | 1448 non-null | object |
| 5 | Neighborhood | 1453 non-null | object |
| 6 | BldgType | 1449 non-null | object |
| 7 | HouseStyle | 1454 non-null | object |
| 8 | YearBuilt | 1453 non-null | float64 |
| 9 | RoofStyle | 1454 non-null | object |
| 10 | Foundation | 1452 non-null | object |
| 11 | Electrical | 1447 non-null | object |
| 12 | GrLivArea | 1445 non-null | float64 |
| 13 | FullBath | 1454 non-null | float64 |
| 14 | BedroomAbvGr | 1452 non-null | float64 |
| 15 | KitchenAbvGr | 1455 non-null | float64 |
| 16 | Fireplaces | 1456 non-null | float64 |
| 17 | GarageCars | 1456 non-null | float64 |
| 18 | SaleCondition | 1459 non-null | object |
| 19 | SalePrice | 1460 non-null | int64 |

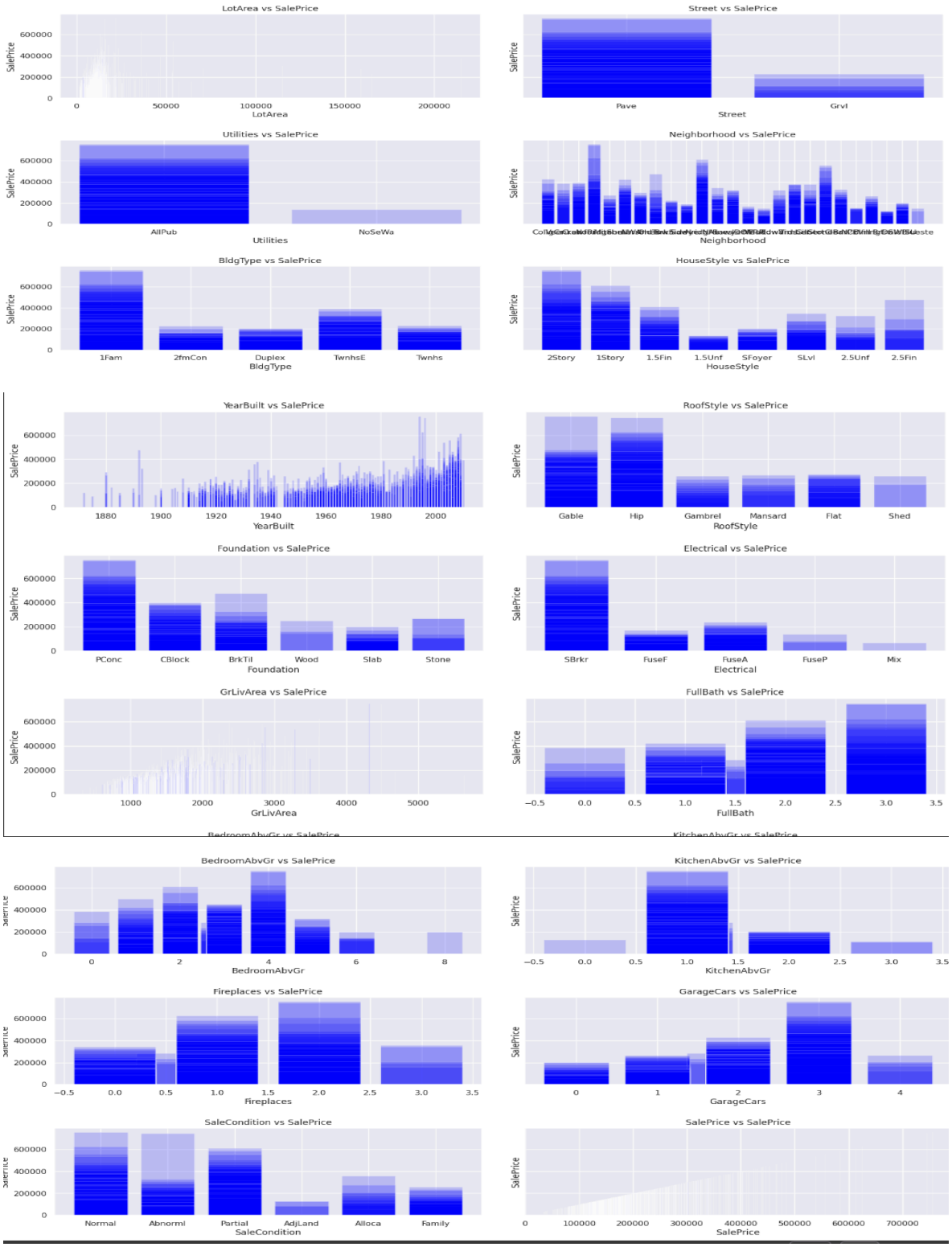
**For correlation between all features, we use the Seaborn library for creating a heatmap, and the following is the correlation graph.**



## Imbalanced Dataset:

Yes, we had an imbalanced datasheet because all unique classes do not have an equal number of Instances.



## Data Pre-Processing

## Solution for NULL Values

**Drop Rows/Columns:**

**Dropping Features having NULL values more than 50% :**

For pre-processing firstly we checked .isnull() to see if there are any null values. Then we checked the total null values of each column with .isnull().sum() . After that we checked if there is any feature with more than 50% null value, which is 'Alley' in our case, so we dropped that feature. On the other hand, there is a column named Id, which is also irrelevant in our point of view that's why we dropped it also.

**Before:**

```
housing.shape
```

```
(1460, 20)
```

```
housing.isnull().sum()
```

```
Id                0
LotArea           1
Street            0
Alley          1369
Utilities        12
Neighborhood      7
BldgType         11
HouseStyle        6
YearBuilt         7
RoofStyle         6
Foundation        8
Electrical       13
GrLivArea        15
FullBath          6
BedroomAbvGr      8
KitchenAbvGr      5
Fireplaces        4
GarageCars        4
SaleCondition     1
SalePrice         0
dtype: int64
```

**After:**

```
[126] housing.drop(['Alley'], axis=1, inplace=True)
      housing.drop(['Id'], axis=1, inplace=True)

      housing.shape
```

```
(1460, 18)
```

```
housing.isnull().sum()
```

```
LotArea           1
Street            0
Utilities        12
Neighborhood      7
BldgType         11
HouseStyle        6
YearBuilt         7
RoofStyle         6
Foundation        8
Electrical       13
GrLivArea        15
FullBath          6
BedroomAbvGr      8
KitchenAbvGr      5
Fireplaces        4
GarageCars        4
SaleCondition     1
SalePrice         0
dtype: int64
```

**Impute Values:**

**Replacing NULL Values of Quantitative Features:**

For the Quantitative features, we replaced the null values with the mean value of that feature using fillna(housing['Column_Name'].mean()).

```
housing['LotArea'] = housing['LotArea'].fillna(housing['LotArea'].mean())
housing['YearBuilt'] = housing['YearBuilt'].fillna(housing['YearBuilt'].mean())
housing['GrLivArea'] = housing['GrLivArea'].fillna(housing['GrLivArea'].mean())
housing['FullBath'] = housing['FullBath'].fillna(housing['FullBath'].mean())
housing['BedroomAbvGr'] = housing['BedroomAbvGr'].fillna(housing['BedroomAbvGr'].mean())
housing['KitchenAbvGr'] = housing['KitchenAbvGr'].fillna(housing['KitchenAbvGr'].mean())
housing['Fireplaces'] = housing['Fireplaces'].fillna(housing['Fireplaces'].mean())
housing['GarageCars'] = housing['GarageCars'].fillna(housing['GarageCars'].mean())
```

```
housing.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 18 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   LotArea        1460 non-null   float64
 1   Street         1460 non-null   object
 2   Utilities      1448 non-null   object
 3   Neighborhood   1453 non-null   object
 4   BldgType       1449 non-null   object
 5   HouseStyle     1454 non-null   object
 6   YearBuilt      1460 non-null   float64
 7   RoofStyle      1454 non-null   object
 8   Foundation     1452 non-null   object
 9   Electrical     1447 non-null   object
 10  GrLivArea      1460 non-null   float64
 11  FullBath       1460 non-null   float64
 12  BedroomAbvGr   1460 non-null   float64
 13  KitchenAbvGr   1460 non-null   float64
 14  Fireplaces     1460 non-null   float64
 15  GarageCars     1460 non-null   float64
 16  SaleCondition  1459 non-null   object
 17  SalePrice      1460 non-null   int64
```

```
housing.isnull().sum()
LotArea          0
Street           0
Utilities       12
Neighborhood     7
BldgType        11
HouseStyle       6
YearBuilt        0
RoofStyle        6
Foundation       8
Electrical      13
GrLivArea        0
FullBath         0
BedroomAbvGr     0
KitchenAbvGr     0
Fireplaces       0
GarageCars       0
SaleCondition    1
SalePrice        0
dtype: int64
```

**Replacing NULL Values of Categorical Features:**

For the Categorical features, we replaced the null values with the mode value(Most Frequent Value) of that feature using fillna(housing['Column_Name'].mode()[0]).

```
housing['Utilities'] = housing['Utilities'].fillna(housing['Utilities'].mode()[0])
housing['Neighborhood'] = housing['Neighborhood'].fillna(housing['Neighborhood'].mode()[0])
housing['BldgType'] = housing['BldgType'].fillna(housing['BldgType'].mode()[0])
housing['HouseStyle'] = housing['HouseStyle'].fillna(housing['HouseStyle'].mode()[0])
housing['RoofStyle'] = housing['RoofStyle'].fillna(housing['RoofStyle'].mode()[0])
housing['Foundation'] = housing['Foundation'].fillna(housing['Foundation'].mode()[0])
housing['Electrical'] = housing['Electrical'].fillna(housing['Electrical'].mode()[0])
housing['SaleCondition'] = housing['SaleCondition'].fillna(housing['SaleCondition'].mode()[0])
```

```
housing.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 18 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   LotArea        1460 non-null   float64
 1   Street         1460 non-null   object
 2   Utilities      1460 non-null   object
 3   Neighborhood   1460 non-null   object
 4   BldgType       1460 non-null   object
 5   HouseStyle     1460 non-null   object
 6   YearBuilt      1460 non-null   float64
 7   RoofStyle      1460 non-null   object
 8   Foundation     1460 non-null   object
 9   Electrical     1460 non-null   object
 10  GrLivArea      1460 non-null   float64
 11  FullBath       1460 non-null   float64
 12  BedroomAbvGr   1460 non-null   float64
 13  KitchenAbvGr   1460 non-null   float64
 14  Fireplaces     1460 non-null   float64
 15  GarageCars     1460 non-null   float64
 16  SaleCondition  1460 non-null   object
 17  SalePrice      1460 non-null   int64
dtypes: float64(8), int64(1), object(9)
memory usage: 205.4+ KB
```

```
housing.isnull().sum()
```

```
LotArea         0
Street          0
Utilities       0
Neighborhood    0
BldgType        0
HouseStyle      0
YearBuilt       0
RoofStyle       0
Foundation      0
Electrical      0
GrLivArea       0
FullBath        0
BedroomAbvGr    0
KitchenAbvGr    0
Fireplaces      0
GarageCars      0
SaleCondition   0
SalePrice       0
dtype: int64
```

### Solution of Categorical Values

As we cannot work with categorical values as most of the models works for numerical values. So, we need to solve this problem by introducing a encoding method. There are many types of encoding method like LabelEncoder but in our case we used One Hot Encoding. Because all the categorical features hold significant amount of unique data and the data doesn't carry any special with its meaning. That's why we used One Hot Encoding, which actually converts each categorical value into a new categorical column and assign a binary value of 1 or 0 to those columns.

```
categorical_cols = ['Street', 'Utilities', 'Neighborhood', 'BldgType',
                    'HouseStyle', 'RoofStyle', 'Foundation', 'Electrical', 'SaleCondition']

housing_encoded = pd.get_dummies(housing, columns=categorical_cols,dtype=int)
housing_encoded.head()
```

**Feature Scaling:** We also scaled the data using StandardScaler.

```
scaler = StandardScaler()

batch_size = 500

for i in range(0, len(housing_encoded), batch_size):
    scaler.partial_fit(housing_encoded[i:i+batch_size])

X_scaled = scaler.transform(housing_encoded)
```

**Dataset Splitting:** We split the train set and test set using the train_test_split method(Random). We used 70% of the data for test set and 30% of the data for the train set.

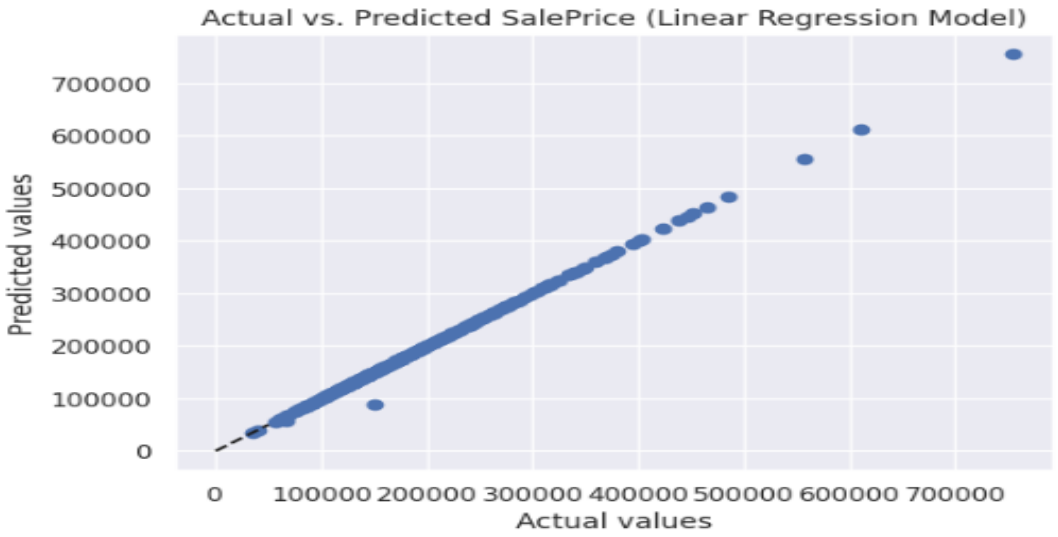
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
```

**Model training & testing**

We used 3 models for training and testing.

**Linear Regression:**

Linear regression is a linear model that assumes a linear relationship between the input variables (x) and the single output variable (y). More specifically, that y can be calculated from a linear combination of the input variables (x). Linear regression fits a linear model with coefficients w = (w1, …, wp) to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation. The first step is to check if all the samples are outliers. If so, it is not possible to fit a linear regression model, and an appropriate message is printed. If no outliers are found, the outliers are removed from the training set, and a linear regression model is fit on the cleaned training data. The model is then used to predict the target variable on the test set, and various evaluation metrics are calculated, such as Mean Squared Error=9127808.41 (MSE), Root Mean Squared Error=3021.23 (RMSE), and Mean Absolute Error = 162.90 (MAE). Additionally, the R-squared score is calculated to measure the goodness of fit of the model. The code then checks for overfitting or underfitting by comparing the training and test accuracies. If the training accuracy is much higher than the test accuracy, the model is overfitting, and if the training accuracy is much lower than the test accuracy, the model is underfitting. Finally, a graph is plotted to show the predicted values versus the actual values to visualize how well the linear regression model fits the data. Overall, this code is a good example of how to perform linear regression on a dataset and evaluate the model's performance, including checking for overfitting or underfitting and visualizing the results as we got accuracy of 99.869193%.

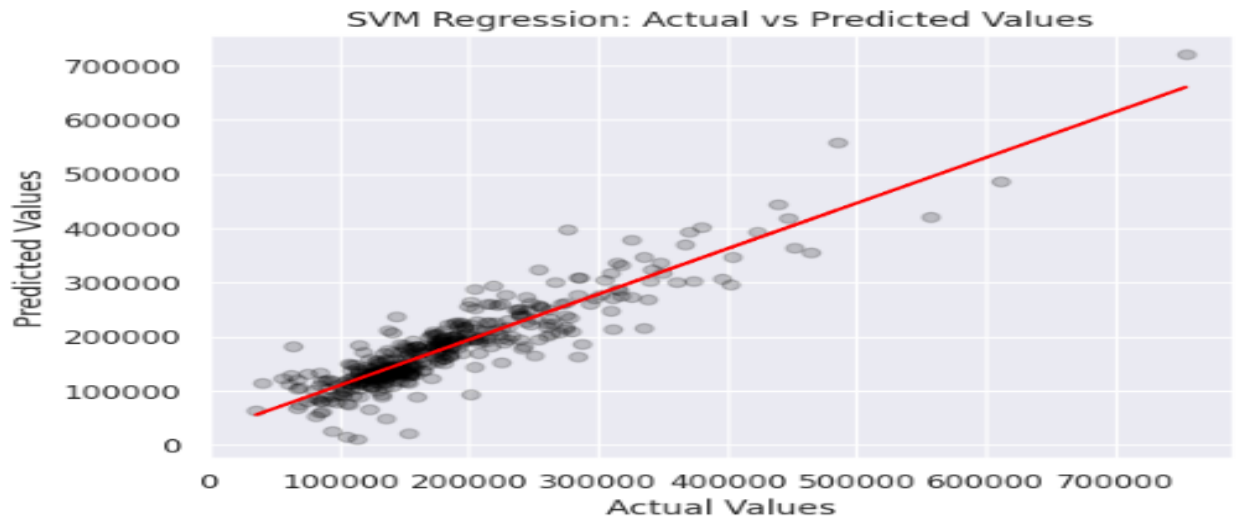


**Decision Tree:**

We used a Decision Tree Regression model to predict the sale price of houses. Firstly, we standardized the feature matrix X using a standardized scaler and split the dataset into a 70% training set and a 30% testing set using the `train_test_split()` method. We then fitted the training data into the Decision Tree Regression model and predicted the target variable on the testing data. The model achieved an accuracy of 99.8575%, as measured by the R-squared score. We also calculated the mean squared error (MSE), root mean squared error (RMSE), and mean absolute error (MAE) to evaluate the model's performance. The MSE and RMSE were 9945534.02 and 3153.65, respectively, while the MAE was 663.12. These values indicate that the model's predictions were fairly accurate. To check for overfitting or underfitting, we calculated the training and testing accuracies, which were both close to 1, indicating that the model was neither overfitting nor underfitting the data. Finally, we visualized the decision tree using the `export_graphviz()` function and the `graphviz` library. The decision tree allows us to see how the model is making predictions by breaking down the dataset into smaller and smaller subsets based on the features. Overall, the Decision Tree Regression model appears to be a good fit for the housing dataset, achieving high accuracy and avoiding overfitting or underfitting.

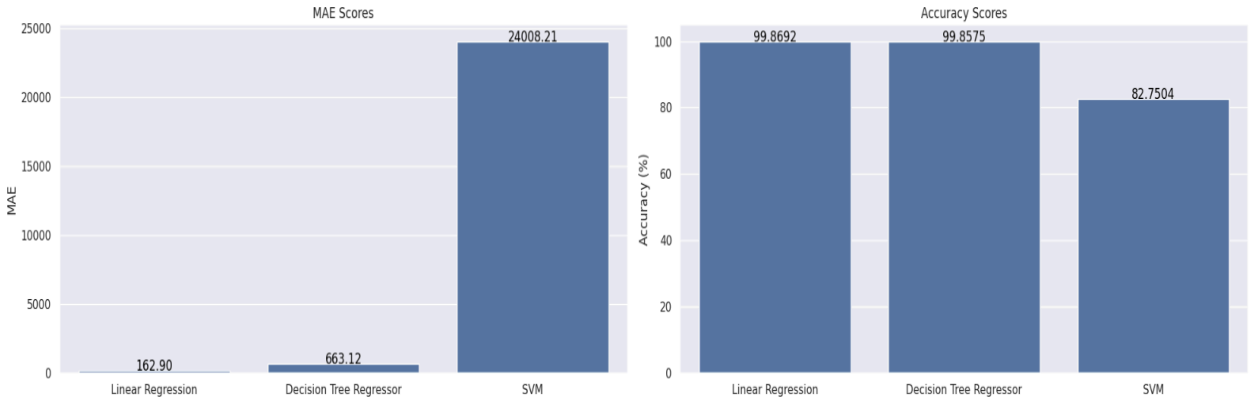
**SVM(Support Vector Machine):**

For the SVM regression model, we used RobustScaler to standardize the feature matrix X before splitting it into training and testing sets using the `train_test_split()` method. We used a support vector regression algorithm with a polynomial kernel function and set the C and coef0 parameters to 1000000 and 0.1. The c parameters control the regularization of the model and the coef is the independent function. We checked for negative predicted values and found none. We then evaluated the performance of the model using the mean squared error (MSE), root mean squared error (RMSE), mean absolute error (MAE), and R-squared score. The model achieved an accuracy of 82.75%, as measured by the R-squared score. We also checked for overfitting or underfitting by calculating the training and testing accuracies, which were 0.85 and 0.82, respectively, indicating that the model was not overfitting or underfitting the data. Finally, we plotted a graph of the actual versus predicted values to visualize how well the model fits the data. The graph shows that the predicted values are generally close to the actual values, although there are some outliers where the predicted values are much lower or higher than the actual values. Overall, the SVM regression model with a polynomial kernel appears to be a good fit for the housing dataset, achieving reasonable accuracy and avoiding overfitting or underfitting.




**Model selection/Comparison analysis:**

And here is our output and comparative analysis part, in the models array we took all the models we used. Also, there is mae and the accuracy score. And this is our final bar chart one is based on mae score and the other is accuracy. As we know the lowest the mae value is the accuracy is higher, and here it also follows that. And finally, this is the final accuracy of our all models.




**Conclusion:**

In conclusion, we have explored three different machine-learning algorithms for estimating the price of a house based on its features and location. We used decision tree, linear regression, and support vector machine (SVM) models to predict the sale price of houses. We pre-processed the data by handling missing values and using one-hot encoding to convert categorical features to numerical ones. We also used StandardScaler to scale the data and split the dataset into a 70% training set and a 30% testing set. After evaluating the performance of each model, we found that the Decision Tree and Linear Regression performed the best, achieving high accuracy and avoiding overfitting or underfitting the data. The SVM also performed reasonably well, but not as well as the decision tree and Linear Regression models.